# AES Encryption

•••

Group C

Andrew Chang
Adam DeVictoria
Rachel Martinez
Nicholas Williams

# Goals

- Implement AES 128, 192, and 256 encryption in C
- Measure execution time for repeated encryptions and calculate average time per encryption
- Compare the difference between encryption times based on block sizes
- Compare the difference between encryption times based on clock speed
- Evaluate performance of AES software implementation

# What is AES?

- Advanced Encryption Standard
- AES encrypts data into 16-byte blocks using a key and a series of rounds that transform the data
- This encryption makes the original data unreadable without the correct key
- Common uses include wireless networking, disk encryption, web security, etc.

# Benchmarking

Benchmarking is the process of measuring system performance by running a task repeatedly and recording execution time.

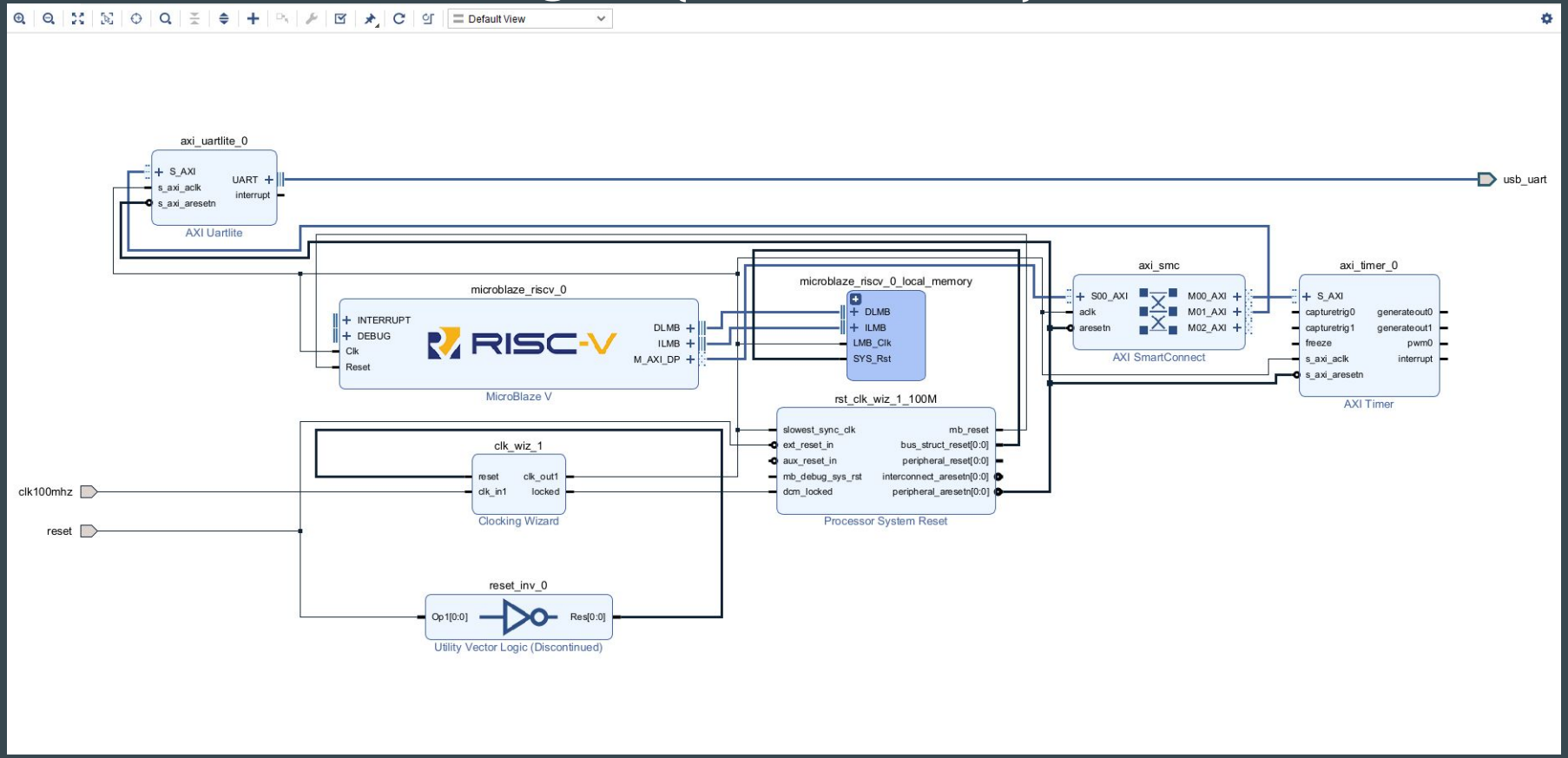
In this project:

- AES encryption is executed many times
- Total runtime is measured
- Average time per encryption is calculated.

# System Setup

- Programming Language: C
- AES library implementation
- Standard test vectors for verification
- Repeated encryption loop for benchmarking

# Microblaze V Block Diagram (Done in Vivado)



# Implementation

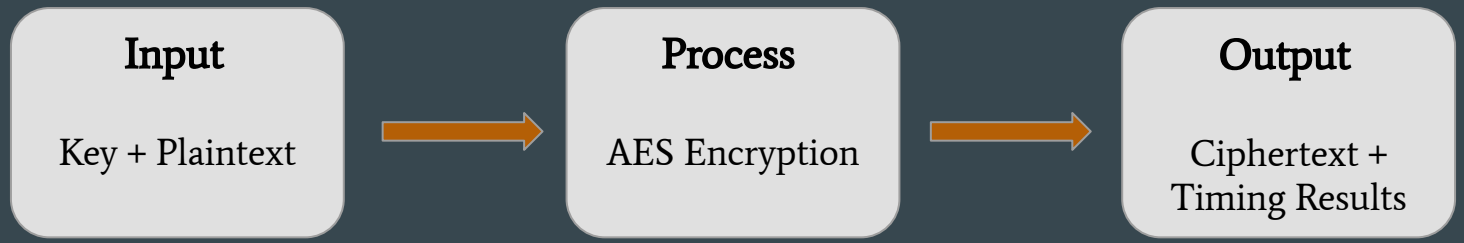
- Define encryption key and plaintext
- Initialize AES context
- Encrypt data block using ECB mode
- Repeat encryption multiple times
- Compare result with expected ciphertext

```c
printf("Testing AES-128\n");
uint8_t key[16] = {
    0x2b,0x7e,0x15,0x16,0x28,0xae,0xd2,0xa6,
    0xab,0xf7,0x15,0x88,0x09,0xcf,0x4f,0x3c
};

// Plaintext
uint8_t plaintext[16] = {
    0x32,0x43,0xf6,0xa8,0x88,0x5a,0x30,0x8d,
    0x31,0x31,0x98,0xa2,0xe0,0x37,0x07,0x34
};
```

```c
uint8_t buffer[16]; //temporary 16 byte
struct AES_ctx ctx; //stores the expand

for (int i = 0; i < numtests; i++) { //
memcpy(buffer, plaintext, 16);
AES_init_ctx(&ctx, key);
AES_ECB_encrypt(&ctx, buffer);
}
```

# Program Flow

- Load key and plaintext
- Initialize AES encryption context
- Encrypt 16-byte data block
- Repeat encryption 1 million times
- Measure total execution time
- Compute average time per encryption



# Output

- Plaintext = original unscrambled text
- Expected = hard-coded expected output
- Ciphertext = output after many rounds of encryptions
- We did 1 million encryptions each for 128, 192, and 256-bit AES using the same plaintext and measured execution time

COM4 - Tera Term VT

File Edit Setup Control Window Help

```
AES test starting...
Testing AES-256
PLAINTEXT  : 6bc1bee22e409f96e93d7e117393172a
CIPHERTEXT : f3eed1bdb5d2a03c064b5a7e3db181f8
EXPECTED   : f3eed1bdb5d2a03c064b5a7e3db181f8
TEST PASSED

Ran 1000000 encryptions

AES test starting...
Testing AES-256
PLAINTEXT  : 6bc1bee22e409f96e93d7e117393172a
CIPHERTEXT : f3eed1bdb5d2a03c064b5a7e3db181f8
EXPECTED   : f3eed1bdb5d2a03c064b5a7e3db181f8
TEST PASSED

Ran 1000000 encryptions

AES test starting...
Testing AES-256
PLAINTEXT  : 6bc1bee22e409f96e93d7e117393172a
CIPHERTEXT : f3eed1bdb5d2a03c064b5a7e3db181f8
EXPECTED   : f3eed1bdb5d2a03c064b5a7e3db181f8
TEST PASSED

Ran 1000000 encryptions

AES test starting...
Testing AES-256
PLAINTEXT  : 6bc1bee22e409f96e93d7e117393172a
CIPHERTEXT : f3eed1bdb5d2a03c064b5a7e3db181f8
EXPECTED   : f3eed1bdb5d2a03c064b5a7e3db181f8
TEST PASSED

Ran 1000000 encryptions
```

# Results

4 trials of each type of encryption

AES-128:

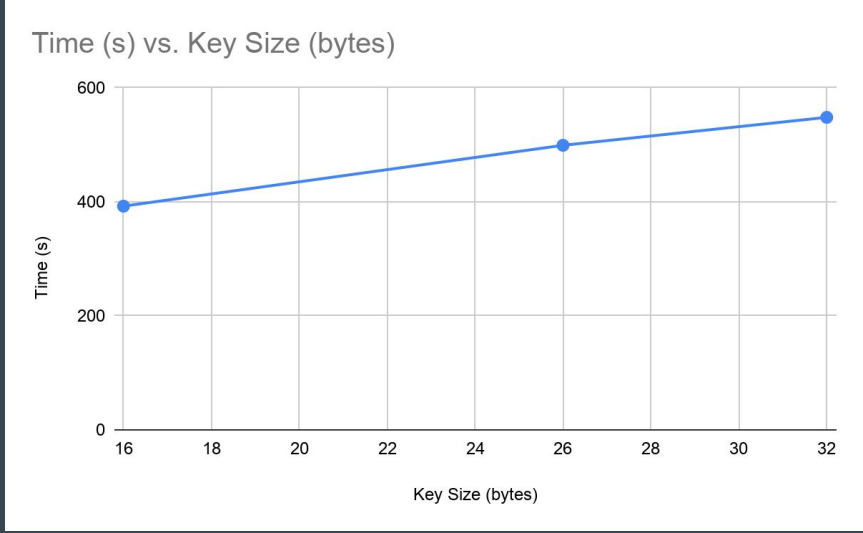
- 392.375 seconds to encrypt
- 2,548.4 encryp/s or 39.8 KB/s

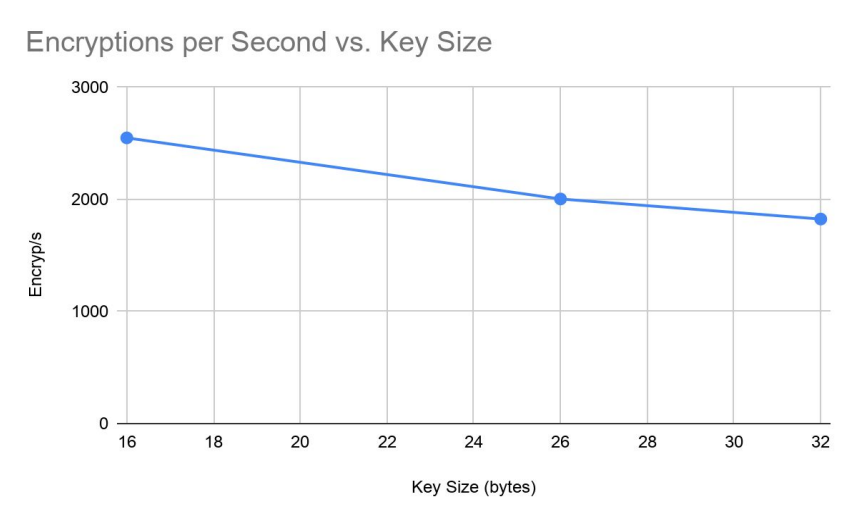
AES-196

- 499.125 seconds to encrypt
- 2,003.6 encryp/s or 31.3 KB/s

AES-256

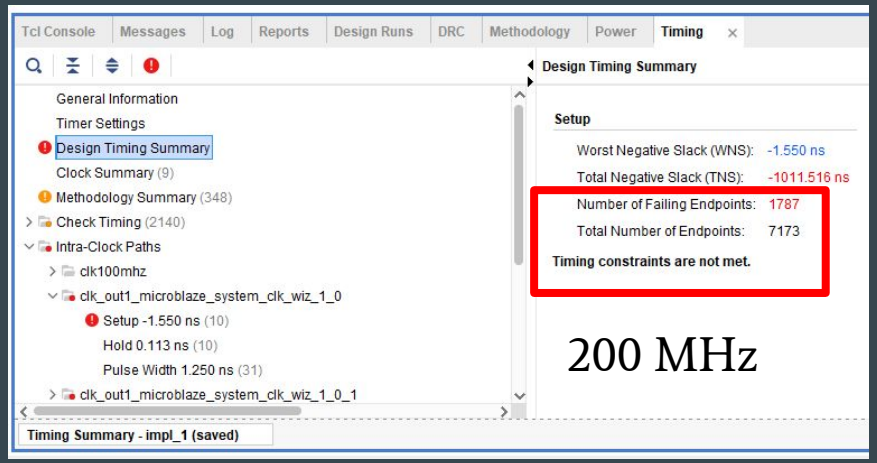
- 548.1325 seconds to encrypt
- 1,824.5 encryp/s or 28.5 KB/s



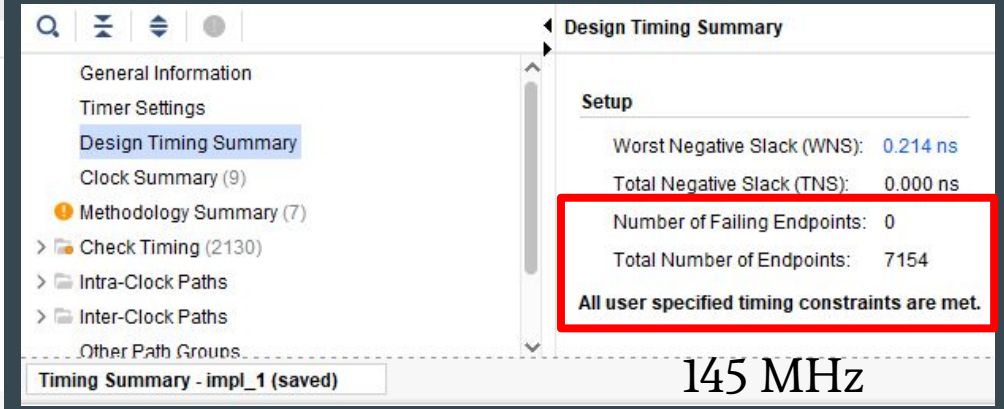



# Alternate Clock Speed

- Base clock speed for Nexys A7-100T is 100 MHz but in Vivado's clocking wizard, the speed can be changed
- We tried to go up to 200 MHz but the implementation timing kept failing
- 145 MHz was the highest clock speed that succeeded
- 50 MHz was the lowest clock speed that we tested




200 MHz




145 MHz

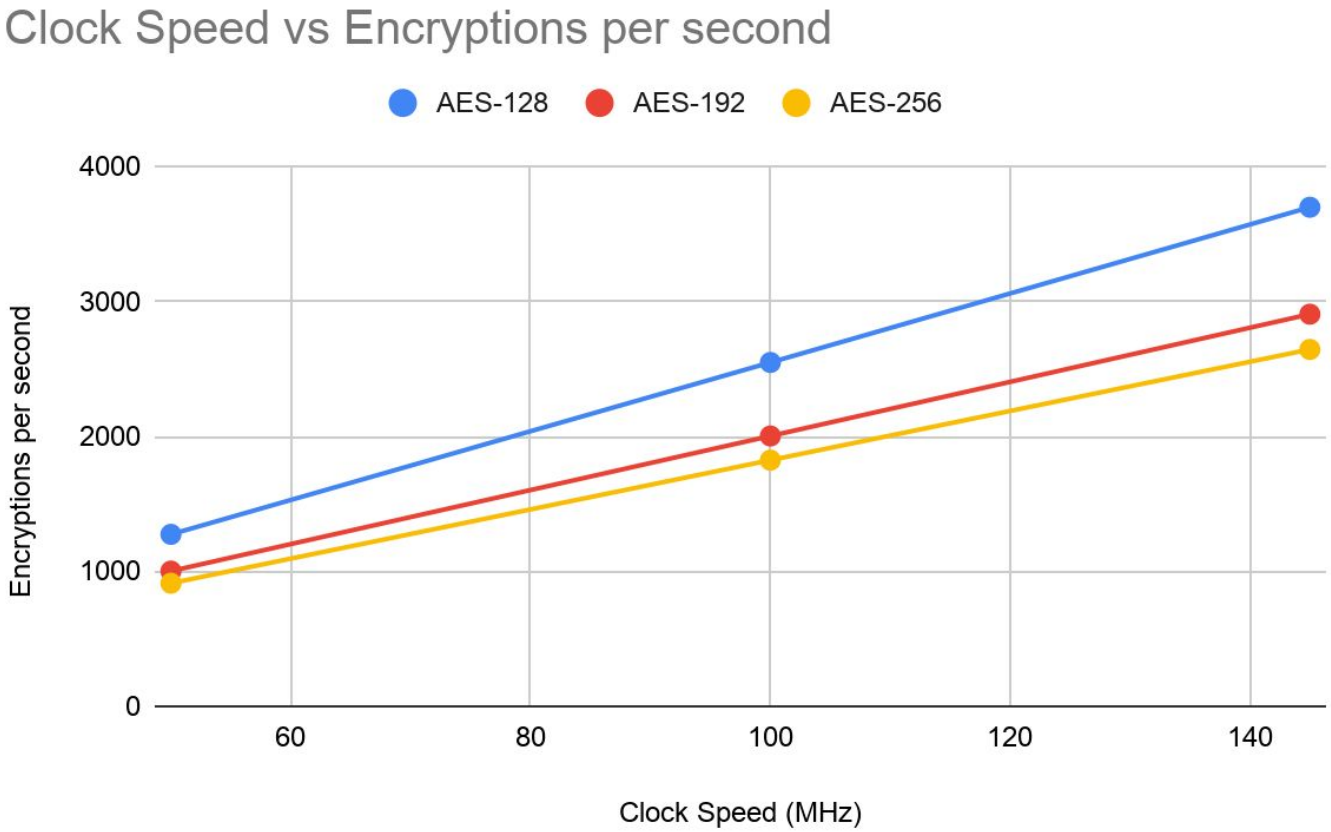
# Comparing Clock Speeds (1 million encryptions)

Time to encrypt

| Clock Speed | AES-128 | AES-192 | AES-256 |
|---|---|---|---|
| 50 MHz | 13:04.46 (784.46s) | 16:37.92 (997.92s) | 18:16.42 (1,096.42s) |
| 100 MHz | 6:32.375 (392.375s) | 8:19.125 (499.125s) | 9:08.133 (548.133s) |
| 145 MHz | 4:30.32 (270.32s) | 5:44.06 (344.06s) | 6:18.05 (378.05s) |

Encryptions/second

| Clock Speed | AES-128 | AES-192 | AES-256 |
|---|---|---|---|
| 50 MHz | 1,274.5 encryp/s | 1,002.1 encryp/s | 912.2 encryp/s |
| 100 MHz | 2,548.6 encryp/s | 2,003.5 encryp/s | 1,824.4 encryp/s |
| 145 MHz | 3,699.31 encryp/s | 2,906.5 encryp/s | 2,645.1 encryp/s |


Clock Speed vs Encryptions per second
AES-128
AES-192
AES-256
4000
3000
2000
1000
0
Encryptions per second
60
80
100
120
140
Clock Speed (MHz)

# Extra Tests (Personal computers, AES-256 10,000,000 encryptions)

AMD Ryzen 5 2600 @ 3.4 GHz (32.3 seconds)
AMD Radeon RX 580
16 GB RAM

Apple M1 @ 3.2 GHz (21.4 seconds)
Apple M1 integrated GPU
8 GB RAM

Intel i7-11800H @ 2.30GHz (21.2 seconds)
Intel UHD Graphics
32 GB RAM

AMD Ryzen 9 7940 @ 4.00GHz (20.54 seconds)
Radeon 780M
16 GB RAM

# Observations

We were unable to isolate singular variables such as amount of RAM, cache, and any hardware AES capability, however:

- Clock speed makes a big difference (100MHz on FPGA vs. 2-4GHz on PCs)
- Our own computers ran the AES encryptions much faster than our FPGA boards
  - About 2.1 seconds for 1 million AES-256 encryptions vs. 548 seconds for FPGA software-only
  - Upscaling FPGA's clock speed from 100 MHz to 3.0 GHz, we would expect 54,732 encryptions/s but our computers did more than 476,000 encryptions/s
- Many CPUs have special hardware dedicated to AES (AES-NI in Intel & AMD CPUs, ARM cryptography extensions, etc. that are much faster than software AES
- Running AES in parallel (such as with GPU acceleration) means multiple blocks of AES can be encrypted at the same time, vs. sequential software AES 1 at a time

# Conclusion

- AES encryption was successfully implemented using C software
- Encryption correctness was verified using known test vectors
- Performance was measured using repeated encryptions and average time per encryption
- Results showed:
  - Increased key sizes lead to slower encryption time
  - Increased clock speeds lead to faster encryption time